

Índice

1. ¿Qué es Ñ?	2
2. Estructura general y sintaxis	3
3. Tipos de datos	3
4. Entrada y salida	4
5. Estructuras de control y ciclos	5
6. Ejemplos	7

¿Qué es Ñ?

Ñ es un lenguaje de programación que fue diseñado con la idea de crear un lenguaje en español y con una sintaxis sencilla para que las personas que van entrando en el mundo de los lenguajes de la programación pudieran iniciar su camino con un lenguaje amigable y en su propio idioma. Hasta el momento (Ñ v2.5) no cuenta con la capacidad de crear funciones ni clases, por lo cual su utilidad es meramente académica y para algoritmos sencillos.

Para más información póngase en contacto con nosotros:

- <http://selenesoft.net/>
- <http://www.facebook.com/selenesoft/>
- <http://selenesoft.net/downloads/compiladores/nn/Soporte>

Estructura general y sintaxis

La apertura del programa es con la palabra **INICIO** que siempre se escribe en mayúsculas y antes de cualquier línea. Para la clausura del programa se usa la palabra **FIN** que se escribe siempre en mayúsculas y después de ella no se escribe ninguna línea de código.

A pesar de que el lenguaje es en español, en ninguna de las palabras de éste lenguaje de programación se utilizan las tildes o acentos (´), esto para minimizar los errores al momento de programar.

La sintaxis de Ñ tal vez recuerde un poco a Visual Basic y C++. Las sentencias se realizan por línea, por lo cual no se pueden colocar 2 sentencias en una misma línea ya que el lenguaje carece de un separador a diferencia de lenguajes como C# o Java que es el “;”.

Ejemplo:

Incorrecto: numerico n = Leer(numerico) Escribir(“El número es: ” + n)

Correcto: numerico n = Leer(numerico)
Escribir(“El numero es: ” + n)

En Ñ no se pueden declarar funciones nuevas, sin embargo el lenguaje cuenta con funciones integradas las cuales inician con mayúscula e incluyen paréntesis después del nombre; algunas de estas funciones aceptan argumentos, como es el caso de **Escribir()**, la cual sirve para mostrar un texto en la consola y se necesita escribir dentro del paréntesis una cadena.

Hay estructuras de en el lenguaje que se necesita especificar dónde inicia el bloque de código y donde termina, como en la estructura **si**.

Tipos de datos

En Ñ solo hay cuatro tipos de datos que son:

- **numerico:** Sirve para almacenar números enteros del -2.147.483.648 a 2.147.483.647.
- **decimal:** Sirve para almacenar desde números enteros y números con punto decimal del $\pm 5,0 \times 10^{-324}$ a $\pm 1,7 \times 10^{30}$
- **caracter:** Almacena un solo carácter (ya sea alfanumérico o especial)
- **cadena:** Almacena un arreglo de caracteres o texto. Para asignar un valor a esta variable se necesita poner entre comillas (“”).
- **Lógico:** Almacena solo 2 valores, **verdadero** o **falso**.

Los tipos de dato siempre van en minúscula y sin acentuación.

A pesar de que Ñ es un lenguaje tipado, no hay funciones que permitan la conversión de un tipo de dato a otro, por lo cual hay que tener cuidado al momento de almacenar el dato de entrada en una variable.

Entrada y salida

La entrada en el lenguaje Ñ es solo por medio del teclado y la salida es solo por medio de la pantalla. A diferencia de los lenguajes tipo C, Ñ contiene 2 funciones para la entrada y salida de datos:

- **Escribir(cadena):** Ésta función recibe un argumento que es el texto que se va a mostrar en pantalla. Si lo que se va a escribir es texto predefinido será necesario usar comillas inglesas (“ ”) al inicio del texto y al terminar, en caso que sea una variable no será necesario escribir comillas al inicio o final del nombre de la variable. Si se desea imprimir varias cadenas se utiliza el símbolo aritmético de adición (+) de esta manera se le indica al computador que debe de unir las cadenas, lo mismo cuando son variables de otro tipo (**numerico**, **decimal** y

caracter) solo que estas no pueden ir solas como argumento, siempre deberán de ir acompañadas de texto y si solo se quiere mostrar el valor de la variable se puede poner solo las comillas (variable + "") para que no haya errores al momento de escribir en pantalla.

Ejemplo:

Incorrecto

```
cadena nombre = "Néstor Rodríguez"
numerico n = 3
Escribir(Hola mundo)
Escribir("nombre")
Escribir("Hola " nombre)
Escribir(n)
```

Correcto

```
cadena nombre = "Néstor Rodríguez"
numerico n = 3
Escribir("Hola mundo") <- de esta manera leerá el argumento como texto predefinido.
Escribir(nombre) <- de esta manera leerá el argumento como una variable.
Escribir("Hola " + nombre) <- Concatenación de un texto predefinido con una variable.
Escribir("El número es: " + n) o Escribir(n + "") <- Concatenación correcta de un texto predefinido con una variable numérica.
```

Leer(tipo de dato)

La entrada de datos en R siempre es por medio del teclado. Ya que en R no hay manera de convertir de un tipo de dato en otro al momento de introducir un dato en la variable se tiene que definir qué tipo de dato leerá.

Ejemplo

```
numerico n = Leer(numerico) <- De esta manera el usuario escribe algo en el teclado y se almacena en la variable como tipo numerico.
cadena c = Leer() o cadena c = Leer(cadena) <- Para recibir un tipo de dato cadena hay 2 maneras, la primera es dejando la función leer sin argumento o escribir el tipo de dato cadena dentro de la función.
```

Excepciones y errores: Si se lee una cadena y se guarda en una variable tipo numérica se producirá un error. Al momento de ejecutarse el programa se puede producir una excepción ya que se puede leer un numérico y guardarse en una variable numérica pero el usuario puede presionar una letra o carácter especial lo cual afectará al flujo del programa.

Estructuras de control y ciclos

En R existe 2 estructuras de control:

si:

Para utilizar esta estructura se escribe la palabra **si** seguido de esta la condición entre paréntesis, después escribiremos la llave de apertura, esto para indicar que es lo que se va a realizar en caso de que la condición sea verdadera, una vez que se escriba el código a ejecutar se cierra con la llave de cierre.

Ejemplo:

```
si (a > b)
{
    Escribir("a es mayor a b")
}
```

Otra parte del **si** es **no**, su estructura es similar solo que aquí no escribiremos la condición si no lo que pasará en caso de que no se cumpla la condición del **si**, se escribe inmediatamente que se cierra el **si**.

Ejemplo:

```
si (a > b)
{
    Escribir("a es mayor a b")
}
no
{
    Escribir("a no es mayor b")
}
```

elegir:

En esta estructura se pueden asignar casos específicos. A diferencia del **si**, dentro de los paréntesis no pondremos una condición sino la variable que vamos a analizar, después abrimos con la llave de apertura pero dentro no escribiremos el código a ejecutar sino los **casos**.

El **caso** se pondrá siempre dentro de **elegir**, después de abrir el **caso** se pondrá el estado de la variable que se espera y seguido de esto dos puntos (:), no es necesario poner llaves ya que se cierra con la palabra **romper**. Hay un caso en el cual no se tiene que asignar un estado es específico llamado **otro** y se utiliza de la misma manera que el **caso**.

Ejemplo:

```
elegir(a)
{
    caso 1:
        Escribir("El numero ingresado es 1")
    romper
    caso 2:
        Escribir("El numero ingresado es 2")
    romper
}
```

```

    caso 3:
        Escribir("El numero ingresado es 3")
    romper

    otro:
        Escribir("Numero desconocido")
    romper
}

```

Hay 3 ciclos en Ñ:

· **mientras**

Este ciclo primero evaluará la condición para iniciar su iteración y se escribe de la siguiente manera:

```

mientras(a > b)
{
    Escribir("A sigue siendo mayor que b")
}

```

Este ciclo seguirá en marcha mientras la condición se cumpla.

· **hacer-mientras**

Este ciclo iniciará sin necesidad que se cumpla la condición, una vez que entra y realiza la operación (a partir de la palabra **hacer**), después evaluará la condición, después de poner la condición entre paréntesis, se coloca un punto y coma (;). El ciclo se escribe de la siguiente manera:

```

numerico c =0;
hacer
{
    Escribir("A sigue siendo mayor que b")
}
mientras(a > b);

```

· **para**

En este ciclo se puede declarar un contador dentro del mismo, la sintaxis sería así

```

para(numérico c = c; c <= 10; c++)
{
    Escribir(c + " - Hola mundo")
}

```

Este ciclo es mejor utilizarlo cuando se conoce las veces que se va a realizar el ciclo.

Ejemplos

Diagrama de flujo	Código en Ñ
	INICIO

	<pre> hacer{ Escribir("Introduzca dos valores distintos") numerico A = Leer(numerico) numerico B = Leer(numerico) }mientras(A == B); si(A > B) { Escribir(A + " Es el mayor") } no { Escribir(B + " Es el mayor") } Pausa() FIN </pre>
	<pre> INICIO numerico A = Leer(numerico) numerico B = Leer(numerico) numerico C = Leer(numerico) si(A > B && A > C) { Escribir(A + " Es el mayor") } No { si(B > A && B > C) { Escribir(B + " Es el mayor") } No { Escribir(C + " Es el mayor") } } Pausa() FIN </pre>

Si tienes problemas con el compilador o tienes sugerencias y/o dudas sobre el lenguaje háznosla saber

<http://selenesoft.net/>

- <http://www.facebook.com/selenesoft/>